8 Kasım 2026

BM201
İleri Programlama
Project RoadMap Template

Proje Ekibi:	**Sinan Gün** — 25118080047 **Hasan Can Bayındır** — 23118080091 **İsmail Can Anlaş** — 24118080098 **Ahmet Kıymık** — 24118080704
Proje İsmi:	**CezAI** *(Yapay Zeka Destekli Otoyol Hız Takip ve Dinamik Ceza Yönetim Sistemi)*
Proje Konusu ve Amacı:	**CezAI**, otoyollarda akan araç trafiğini yüksek çözünürlüklü kamera akışları üzerinden gerçek zamanlı bilgisayarlı görü ve derin öğrenme (Computer Vision / Deep Learning) modelleriyle takip eden, araçların anlık hızlarını piksel-metre kalibrasyonuyla tespit eden, yasal hız limitini aşan araçlara aşım oranına göre kademeli ceza kesen ve tüm denetim verilerini ilişkisel veritabanında saklayan akıllı bir trafik denetleme ve yönetim sistemidir. **Projenin Temel Amacı ve Çözdüğü Problem:** Geleneksel radar ve hız tespit sistemleri yüksek donanım maliyetine sahip olup yalnızca tek bir noktadan anlık ölçüm yapabilmektedir. CezAI, mevcut otoyol ve mobese kameralarını yapay zeka ile akıllandırarak ek sensör maliyetine gerek duymaksızın araçları sürekli takip eder. Sistem, araç tipine (otomobil, kamyon, otobüs vb.) göre değişen hız limitlerini otomatik uygular, aşım yüzdesine göre dinamik ceza tutarını hesaplar ve trafik ekiplerinin kullanımına hazır denetim raporları sunar. **Sistem Kullanıcı Rolleri ve Yetkileri:** Sistemde iki temel kullanıcı rolü tanımlanmıştır: **1. Sistem Yöneticisi (Admin):** Kamera akışlarını ve video kaynaklarını sisteme bağlama, otoyol hız sınırı parametrelerini (araç tipi bazlı limitler, hava koşulu toleransları) yapılandırma, operatör hesaplarını yönetme ve genel istatistik panelini (ortalama otoyol hızları, en sık ihlal yapılan saatler, toplam ceza tutarları) inceleme. **2. Trafik Operatörü / Denetçi:** Canlı takip ekranını izleme, anlık ihlal alarmlarını görüntüleme, ihlal anı kanıt fotoğraflarını inceleme, plakaya göre geçmiş ihlal sorgulama ve resmi ceza tutanağı (PDF) oluşturma. **Dinamik Hız Aşımı ve Kademeli Ceza Hesaplama Motoru:** Türkiye Karayolları Trafik Kanunu standartlarına uygun olarak kademeli ceza algoritması kurgulanmıştır: • *Hız Aşım Oranı (ΔV) = [(Ölçülen Hız - Hız*

	Sınırı) / Hız Sınırı] × 100 • **Tolerans (%0 - %10):** Yasal tolerans sınırıdır; ceza uygulanmaz. • **1. Kademe Aşım (%10 - %30):** Düşük düzey para cezası uygulanır. • **2. Kademe Aşım (%30 - %50):** Orta düzey para cezası + ceza puanı işlenir. • **3. Kademe Aşım (%50 ve üzeri):** Ağır para cezası + ehliyete geçici el koyma uyarısı oluşturulur. • **Araç Tipi Çarpanı:** Ağır vasıta (kamyon/otobüs) hız aşımlarında güvenlik riski katsayısı uygulanarak ceza tutarı dinamik olarak güncellenir. **Nesne Yönelimli Programlama (OOP) Prensiplerinin Uygulanması:** • **Encapsulation (Kapsülleme):** Tüm araç, ihlal ve kullanıcı sınıflarında veriler private alanlarda tutulur; hız, aşım oranı ve koordinat değerleri getter/setter metotlarında mantıksal doğrulama (örn: hız negatif olamaz, koordinat ekran dışına taşamaz) kurallarıyla korunur. • **Inheritance (Kalıtım):** Soyut 'Vehicle' üst sınıfından 'Car', 'Truck', 'Bus' ve 'Motorcycle' sınıfları türer. Her araç alt sınıfı kendi otoyol yasal hız sınırını ve ceza çarpanını taşır. Benzer şekilde soyut 'User' sınıfından 'Admin' ve 'Operator' türer. • **Polymorphism (Çok Biçimlilik):** Her araç türü 'CalculatePenalty()' ve 'GetSpeedLimit()' metotlarını kendi kurallarına göre override eder. Ceza hesaplama stratejileri çalışma zamanında (runtime) araç tipine göre polimorfik olarak tetiklenir. • **Abstraction (Soyutlama):** Sistem karmaşıklığını gizlemek için 'ITrackingEngine' (araç tespit ve takip arayüzü), 'IPenaltyStrategy' (ceza hesaplama arayüzü), 'IRepository' (veritabanı CRUD arayüzü) ve 'INotificationService' arayüzleri tanımlanmıştır. **Tasarım Kalıpları (Design Patterns):** • **Strategy Pattern:** Farklı hız aşım kademeleri ve araç türlerine göre dinamik ceza hesaplama algoritmalarını soyutlamak için kullanılır ('StandardPenaltyStrategy', 'HeavyVehiclePenaltyStrategy'). • **Observer Pattern:** Bir aracın hız sınırını aştığı tespit edildiği anda tetiklenen olay yapısıdır. Hız motoru ihlali yayınlar; Veritabanı Kaydedici, Arayüz Uyarı Paneli ve Anlık Ekran Görüntüsü Alıcı servisleri eşzamanlı olarak uyarılır. • **Factory Pattern:** Yapay zeka modelinden gelen nesne sınıf etiketine (class ID: car, truck, bus) göre uygun araç nesnesini ('VehicleFactory') dinamik olarak üretmek için kullanılır. • **Repository Pattern:** Veritabanı işlemlerini iş mantığından tamamen soyutlayarak veri erişimini merkezi ve esnek bir yapıya kavuşturur.

	Planlanan Sınıf Yapısı ve Hiyerarşisi: • **Vehicle (abstract):** Ortak araç alanları (track_id, vehicle_type, bounding_box, current_speed, speed_history, entry_time) • **Car, Truck, Bus, Motorcycle (extends Vehicle):** Türe özgü yasal limitler ve ceza katsayıları • **SpeedTracker:** Kareler arası piksel yer değiştirmesini ve homografi matrisini kullanarak hız hesaplayan ana motor • **ViolationRecord:** İhlal kaydı varlığı (violation_id, vehicle_id, vehicle_type, measured_speed, speed_limit, excess_ratio, fine_amount, timestamp, evidence_image_path) • **PenaltyCalculator:** Aşım oranına ve ceza stratejisine göre parasal tutar belirleyici • **User (abstract), Admin, Operator:** Kullanıcı yönetimi ve yetki seviyeleri • **DatabaseManager / ViolationRepository:** ORM tabanlı veritabanı CRUD operasyonları • **ITrackingEngine, IPenaltyStrategy, IRepository, INotificationObserver:** Sistem arayüzleri
Proje Tasarımı:	**Programlama Dili:** Python (3.11+). Yapay zeka, bilgisayarlı görü ve veri bilimi kütüphaneleriyle yerel ekosistem uyumluluğu, yüksek hesaplama performansı ve zengin nesne yönelimli programlama (OOP) desteği nedeniyle proje ana dili olarak seçilmiştir. **Yapay Zeka ve Bilgisayarlı Görü Motoru:** • **YOLOv8 (Ultralytics):** Otoyol video akışında araç tespiti ve sınıflandırması (otomobil, kamyon, otobüs, motosiklet) için en güncel ve yüksek FPS'li derin öğrenme modeli. • **ByteTrack / DeepSORT:** Tespit edilen araçların kareler boyunca izini kaybetmeden takip edilmesi (Multi-Object Tracking - MOT) ve her araca benzersiz bir takip ID'si atanması için. • **OpenCV:** Görüntü ön işleme, video akışı yakalama, koordinat dönüşümleri ve perspektif düzeltme (bird's-eye view) işlemleri için. **Hız Hesaplama ve Kamera Kalibrasyonu Metodolojisi:** Kamera perspektif bozulmasını gidermek amacıyla otoyol üzerindeki bilinen referans noktaları (şerit çizgileri arası mesafe) kullanılarak Homografi Dönüşümü (Perspective Transform) uygulanır. Araçların video kareleri arasındaki piksel hareketleri gerçek metre mesafesine dönüştürülür. Kamera FPS değeri ile zaman farkı (Δt) hesaplanarak aracın anlık hızı ($v = \Delta x / \Delta t$) yüksek doğrulukla ve gürültü azaltıcı filtrelerle (Moving Average Filter) belirlenir. **Kullanıcı Arayüzü (GUI / Dashboard):** • **PyQt6 / CustomTkinter:** Canlı video akışının oynatıldığı, araçların üzerine hız etiketlerinin ve renk kodlu takip kutularının (hız sınırını aşanlara anında

	kırmızı uyarı kutusu) çizildiği modern masaüstü denetim arayüzü. • **Raporlama & Analiz Paneli:** Anlık ihlal tablosu, filtreleme araçları (tarih, hız, ceza tutarı), ihlal anı kanıt fotoğrafı görüntüleyicisi ve PDF ceza tutanağı oluşturma modülü. **Veritabanı ve ORM:** • **SQLite / PostgreSQL:** İhlal kayıtları, araç detayları, ceza logları ve kullanıcı profillerinin güvenli ve ilişkisel yapıda saklanması için. • **SQLAlchemy ORM:** Veritabanı tablolarının nesne yönelimli sınıflarla eşleştirilmesini ve tip güvenli sorgular yazılmasını sağlar. **Yazılım Mimarisi:** Katmanlı Mimari (N-Tier Architecture / MVC). Presentation Layer (PyQt6 UI), Business Logic Layer (Vehicle & Penalty Engines), Vision Layer (YOLO + Speed Tracker) ve Data Access Layer (SQLAlchemy Repository) katmanları birbirinden bağımsız ve gevşek bağlı (loosely coupled) tasarlanacaktır. **Geliştirme Ortamı ve Sürüm Kontrolü:** PyCharm / VS Code IDE, Git & GitHub (takım içi modüler dal / branch yönetimi).
Proje Yol Haritası (Roadmap):	**Haftalık Çalışma ve Kilometre Taşları Planı (10 Haftalık Çizelge):** • **1-2. Hafta (Gereksinim Analizi & Veri Temini):** Otoyol trafik video setlerinin temin edilmesi, kamera açısı ve şerit mesafe ölçümlerinin çıkarılması, sistem mimarisi ve UML sınıf diyagramlarının hazırlanması. • **3-4. Hafta (Yapay Zeka & Araç Takip Motoru):** YOLOv8 araç algılama modelinin kurulması, ByteTrack takip algoritmasının entegre edilmesi ve araçlara stabil ID atamasının sağlanması. • **5-6. Hafta (Hız Ölçüm Algoritması & Kalibrasyon):** Kamera perspektif düzeltme (homografi) matrisinin çıkarılması, piksel-metre dönüşümü ile anlık hız ölçümünün kodlanması ve gürültü filtreleme testleri. • **7. Hafta (OOP Hiyerarşisi & Dinamik Ceza Motoru):** Vehicle abstract sınıfı, Car/Truck/Bus kalıtım yapısı, Strategy Pattern tabanlı kademeli ceza hesaplama motorunun kodlanması. • **8. Hafta (Veritabanı & Observer Entegrasyonu):** SQLAlchemy ORM modellerinin kurulması, hız aşımı gerçekleştiğinde veritabanına ihlal kaydı ve kanıt karesi kaydeden Observer yapısının bağlanması. • **9. Hafta (Kullanıcı Arayüzü / Dashboard):** PyQt6 ile canlı video takip ekranının, hız kutucuklarının, ihlal bildirim panelinin ve ceza sorgulama ekranlarının geliştirilmesi. • **10. Hafta (Test, Optimizasyon & Teslim):** Farklı araç yoğunluklarında FPS ve doğruluk testleri, uçtan uca hata ayıklama, proje raporu ve final sunumunun

	tamamlanması.